

Metody modelowania matematycznego: Projektu nr 8

Symulacja Numeryczna Dwumasowego Układu Mechanicznego z Przekładnią

Marcin Piechowski 203889 i Szymon Ciarkowski 203770

6 czerwca 2026

Spis treści

1	Przedstawienie problemu	2
2	Część analityczna – wyprowadzenie modelu matematycznego	2
2.1	Związki kinematyczne i transmitowane momenty	2
2.2	Równania różniczkowe ruchu mas wirujących	3
2.3	Redukcja układu do jednej współrzędnej uogólnionej	3
2.4	Sformułowanie równań w przestrzeni stanów	3
3	Metody numeryczne i opis algorytmów	4
3.1	Jawna metoda Eulera (rzędu 1)	4
3.2	Klasyczna metoda Rungego-Kutty czwartego rzędu (RK4)	4
4	Najważniejsze części kodu	5
4.1	Architektura obiektowa	5
4.2	Implementacja algorytmów całkujących	5
5	Wyniki symulacji i ocena jakościowa algorytmów	6
6	Wnioski końcowe	7

1 Przedstawienie problemu

Przedmiotem projektu jest analiza analityczna oraz opracowanie środowiska symulacyjnego dla obrotowego układu mechanicznego. Układ ten reprezentuje model napędu dwumasowego połączonych przekładnią zębatą (redukcyjną lub multiplikującą) wraz z uwzględnieniem elastyczności wału wyjściowego oraz tłumienia lepkościowego.

W skład modelowanej struktury fizycznej wchodzi następujące komponenty składowe:

1. Wał wejściowy (pierwszy): opisany jest skupionym momentem bezwładności J_1 . Na wał ten działa zewnętrzny, sterowany moment napędowy stanowiący wymuszenie $u(t) = \tau_m$. Położenie kątowe tego wału określa współrzędna θ_1 .
2. Układ przekładni zębatej: sprzęga oba wały. Koła zębate opisane są przez parametry n_1 oraz n_2 , które odpowiadają liczbie zębów poszczególnych kół.
3. Wał wyjściowy (drugi): opisany jest skupionym momentem bezwładności J_2 i położeniem kątowym θ_2 . Wzdłuż tego wału występuje oddziaływanie elementów pasywnych: tłumika o współczynniku tłumienia lepkościowego b oraz sprężyny skrętnej o współczynniku sztywności k utwierdzonej do podłoża.

Główne cele projektu obejmują:

1. Wyprowadzenie pełnego matematycznego modelu dynamicznego układu przy wykorzystaniu zasad mechaniki klasycznej.
2. Sprowadzenie równań do postaci równań stanu w celu umożliwienia implementacji numerycznej.
3. Zaimplementowanie symulatora w języku Python pozwalającego na wymuszenie układu trzema rodzajami sygnałów: prostokątnym o skończonym czasie trwania, trójkątnym oraz harmonicznym.
4. Przeprowadzenie całkowania numerycznego dwoma metodami: jawną metodą Eulera oraz klasyczną metodą Rungego-Kutty czwartego rzędu (RK4).
5. Porównanie wyników działania obu algorytmów wraz z oceną jakościową zastosowanych metod numerycznych.

2 Część analityczna – wyprowadzenie modelu matematycznego

W celu sformułowania ścisłych równań dynamicznych opisujących ruch układu, zastosowano metodę rozcinania więzów i niezależnego bilansowania momentów sił dla obu mas rotacyjnych.

2.1 Związki kinematyczne i transmitowane momenty

Zakładamy idealny model przekładni zębatej, w którym pomija się luzy międzyzębne, tarcie na stykach oraz momenty bezwładności samych kół zębatych. Sztywne więzy narzucone przez geometrię kół zębatych determinują jednoznaczną relację pomiędzy kątami obrotu obu wałów:

$$\theta_1 \cdot n_1 = \theta_2 \cdot n_2 \implies \theta_1 = \left(\frac{n_2}{n_1}\right) \cdot \theta_2 \quad (1)$$

Definiując przełożenie mechaniczne przekładni jako $i = \frac{n_2}{n_1}$, możemy zapisać związki dla położenia, prędkości kątowej oraz przyspieszenia kątowego wału wejściowego w funkcji zmiennych wału wyjściowego:

$$\theta_1 = i \cdot \theta_2, \quad \dot{\theta}_1 = i \cdot \dot{\theta}_2, \quad \ddot{\theta}_1 = i \cdot \ddot{\theta}_2 \quad (2)$$

Z zasady zachowania energii lub równowagi mocy układu idealnego wynika, że momenty sił reakcji wewnętrznych na przekładni, oznaczane odpowiednio jako M_{r1} (moment obciążający wał 1 od strony przekładni) oraz M_{r2} (moment napędzający wał 2 od strony przekładni), spełniają zależność:

$$\frac{M_{r2}}{M_{r1}} = \frac{n_2}{n_1} = i \implies M_{r2} = i \cdot M_{r1} \quad (3)$$

2.2 Równania różniczkowe ruchu mas wirujących

Stosując drugą zasadę dynamiki Newtona dla ruchu obrotowego każdej z wydzielonych mas składowych układu, uzyskujemy:

Dla pierwszego wału (o bezwładności J_1):

$$J_1 \ddot{\theta}_1 = u(t) - M_{r1} \quad (4)$$

Gdzie $u(t) = \tau_m$ to zewnętrzny moment napędowy, a M_{r1} to przeciwdziałający moment reakcji przekładni.

Dla drugiego wału (o bezwładności J_2), na który oddziałują dodatkowo momenty oporowe sprężyny $M_k = k\theta_2$ oraz tłumika lepkościowego $M_b = b\dot{\theta}_2$:

$$J_2 \ddot{\theta}_2 = M_{r2} - b\dot{\theta}_2 - k\theta_2 \quad (5)$$

2.3 Redukcja układu do jednej współrzędnej uogólnionej

W celu eliminacji nieznanymi wewnętrznych momentów reakcji przekładni (M_{r1}, M_{r2}), wyznaczamy M_{r1} z równania wału pierwszego i podstawiamy do zależności wiążącej momenty:

$$M_{r1} = u(t) - J_1 \ddot{\theta}_1 \implies M_{r2} = i \cdot (u(t) - J_1 \ddot{\theta}_1) \quad (6)$$

Wprowadzając uzyskane wyrażenie na M_{r2} do równania ruchu wału drugiego, otrzymujemy jedno spójne równanie różniczkowe drugiego rzędu:

$$J_2 \ddot{\theta}_2 = i \cdot u(t) - i^2 \cdot J_1 \ddot{\theta}_2 - b\dot{\theta}_2 - k\theta_2 \quad (7)$$

Po przeniesieniu członów zawierających przyspieszenie na lewą stronę i pogrupowaniu współczynników, model przyjmuje postać:

$$(J_2 + i^2 J_1) \ddot{\theta}_2(t) + b\dot{\theta}_2(t) + k\theta_2(t) = i \cdot u(t) \quad (8)$$

Wprowadzamy pojęcie **zredukowanego momentu bezwładności** układu odniesionego do osi wału wtórnego, zdefiniowanego jako $J_{eq} = J_2 + i^2 J_1 = J_2 + \left(\frac{n_2}{n_1}\right)^2 J_1$. Ostateczne równanie przyjmuje kanoniczną postać oscylatora harmonicznego z tłumieniem:

$$J_{eq} \ddot{\theta}_2(t) + b\dot{\theta}_2(t) + k\theta_2(t) = \left(\frac{n_2}{n_1}\right) u(t) \quad (9)$$

2.4 Sformułowanie równań w przestrzeni stanów

Na potrzeby algorytmów całkujących numerycznie definiujemy wektor stanu jako $x(t) = [x_1(t), x_2(t)]^T$, gdzie pierwszą zmienną stanu jest położenie katowe wału wtórnego $x_1 = \theta_2$, a drugą jego prędkość katowa $x_2 = \dot{\theta}_2 = \omega_2$. Układ równań stanu pierwszego rzędu przyjmuje postać macierzową $\dot{x} = Ax + Bu$:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{J_{eq}} & -\frac{b}{J_{eq}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{n_2}{n_1 J_{eq}} \end{bmatrix} u(t) \quad (10)$$

Definiujemy zatem wektor wyjść jako:

$$y(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \end{bmatrix} = \begin{bmatrix} \theta_2(t) \\ \omega_2(t) \end{bmatrix} \quad (11)$$

Wykorzystując definicję zmiennych stanu ($x_1 = \theta_2$, $x_2 = \omega_2$), wiążemy sygnały wyjściowe bezpośrednio ze zmiennymi stanu oraz wymuszeniem zewnętrznym:

$$y_1 = 1 \cdot x_1 + 0 \cdot x_2 + 0 \cdot u(t) \quad (12)$$

$$y_2 = 0 \cdot x_1 + 1 \cdot x_2 + 0 \cdot u(t) \quad (13)$$

Zapisując powyższe relacje w postaci macierzowej, otrzymujemy ostateczną formę równania wyjścia układu:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u(t) \quad (14)$$

3 Metody numeryczne i opis algorytmów

Do rozwiązania powyższego zagadnienia początkowego Cauchy'ego zastosowano dwa algorytmy o różnej dokładności i stabilności numerycznej.

3.1 Jawna metoda Eulera (rzędu 1)

Jest to podstawowy algorytm aproksymacji pochodnej ilorazem różnicowym w przód. Wartość stanu w kolejnym kroku czasowym $t_{n+1} = t_n + \Delta t$ zależy wyłącznie od wartości w kroku bieżącym:

$$x(t_{n+1}) = x(t_n) + \Delta t \cdot f(t_n, x(t_n)) \quad (15)$$

Gdzie $f(t, x) = \dot{x}$. Globalny błąd obcięcia tej metody wynosi $O(\Delta t)$. Algorytm charakteryzuje się bardzo wąskim obszarem stabilności numerycznej, przez co łatwo ulega rozbieganiu przy symulacji obiektów o charakterze drgającym.

3.2 Klasyczna metoda Rungego-Kutty czwartego rzędu (RK4)

Jest to zaawansowana metoda jednoetapowa wielokrotnego próbkowania (czteroetapowa). W każdym dyskretnym kroku czasowym oblicza się cztery wektory przyrostów cząstkowych, co pozwala uzyskać globalny błąd zbieżności rzędu $O(\Delta t^4)$:

$$k_1 = f(t_n, x_n) \quad (16)$$

$$k_2 = f\left(t_n + \frac{\Delta t}{2}, x_n + \frac{\Delta t}{2}k_1\right) \quad (17)$$

$$k_3 = f\left(t_n + \frac{\Delta t}{2}, x_n + \frac{\Delta t}{2}k_2\right) \quad (18)$$

$$k_4 = f(t_n + \Delta t, x_n + \Delta tk_3) \quad (19)$$

Ostateczna wartość stanu wyznaczana jest jako średnia ważona:

$$x_{n+1} = x_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (20)$$

4 Najważniejsze części kodu

4.1 Architektura obiektowa

Środowisko symulacyjne opiera się na dwóch głównych klasach:

- **Klasa StateSpaceSystem** – pełni rolę obiektu modelu stanowego. Przechowuje macierze stanu i implementuje równania dynamiki układu. Główne metody to:
 - `__init__` – inicjalizuje układ na podstawie macierzy A, B, C i D, weryfikuje zgodność wymiarów podanych macierzy.
 - `get_state_change` – zwraca wektor pochodnych stanu ($\dot{x}$) dla danej chwili. Jest to bezpośrednia implementacja równania stanu w postaci macierzowej.
 - `get_output` – wylicza aktualne wyjście układu (y) na podstawie bieżącego wektora stanu.
- **Klasa SystemSimulation** – główny silnik symulacyjny, który zarządza czasem, zapisuje historię stanów i wywołuje algorytmy całkujące. Najważniejsze metody to:
 - `run` – główna pętla symulacji, iterująca po zadanym czasie. Jej rzeczywisty krok to wywołanie funkcji `step`.
 - `step` – wykonuje pojedynczy krok symulacji. Zaimplementowano w niej mechanizm adaptacji długości kroku całkowania (zagęszczanie siatki czasu w przypadku dużych błędów obciążenia i analogiczne zwiększanie kroku w przypadku małych błędów)
 - `_calc_step` – metoda pomocnicza służąca do wyboru odpowiedniego algorytmu kroku (Euler lub RK4).

4.2 Implementacja algorytmów całkujących

Samo rozwiązywanie algorytmów odbywa się poza klasami, w dedykowanych funkcjach. Obie funkcje przyjmują referencję do metody obliczającej pochodne (`state_change`), co czyni je uniwersalnymi.

```
1 def rk4(state_change: Callable, current_state: Vector, control_input: Signal,
2         step_size: float) -> Vector:
3     k1 = state_change(control_input, current_state)
4     k2 = state_change(control_input, current_state + k1 * (step_size / 2))
5     k3 = state_change(control_input, current_state + k2 * (step_size / 2))
6     k4 = state_change(control_input, current_state + k3 * step_size)
7
8     weighted_slope = ( k1 + 2 * k2 + 2 * k3 + k4 ) / 6
9
10    return current_state + weighted_slope * step_size
11
12 def euler(state_change: Callable, current_state: Vector, control_input: Vector,
13           step_size: float) -> Vector:
14     return current_state + step_size * state_change(control_input, current_state)
```

Listing 1: Implementacja algorytmów numerycznych: jawnej metody Eulera oraz RK4

Metoda Eulera wylicza nowy stan przesuwając się po prostej stycznej. Jej prostota jest w tym projekcie traktowana jako punkt odniesienia, ponieważ w układach słabo tłumionych metoda ta szybko generuje błędy i pompuje sztuczną energię do układu. Stąd elastyczny krok w pętli symulacyjnej

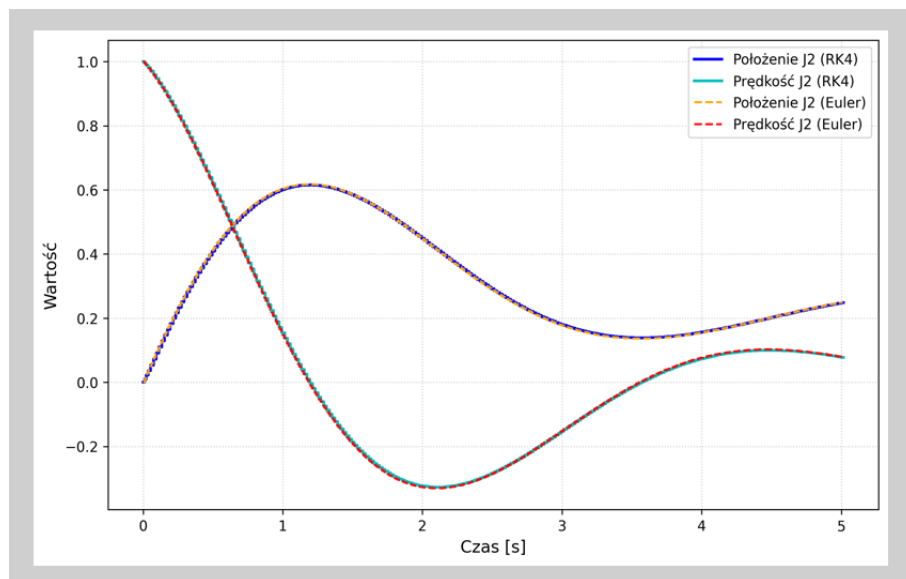
Metoda RK4 Wewnątrz jednego kroku czasowego funkcja wyznacza cztery wagi ($k_1 \dots k_4$), gdzie największy nacisk (mnożnik 2) kładziony jest na próbki z wnętrza przedziału całkowania. Uśrednione nachylenie (`weighted_slope`) pozwala precyzyjnie podążać za nieliniowymi zmianami dynamiki bez utraty stabilności.

5 Wyniki symulacji i ocena jakościowa algorytmów

Weryfikację przeprowadzono za pomocą dwóch różnych testów:

Układ stabilny z silnym tłumieniem z parametrami:

- Sygnał wejściowy: Prostokątny
- Amplituda: 1
- Okres: 10
- Czas trwania symulacji: 5
- Rozmiar kroku: 0.05
- Wartość J_1 : 1
- Wartość J_2 : 1
- Liczba zębów przekładni 1 (n_1): 1
- Liczba zębów przekładni 2 (n_2): 1
- Sztywność (k): 4
- Współczynnik tłumienia (b): 2

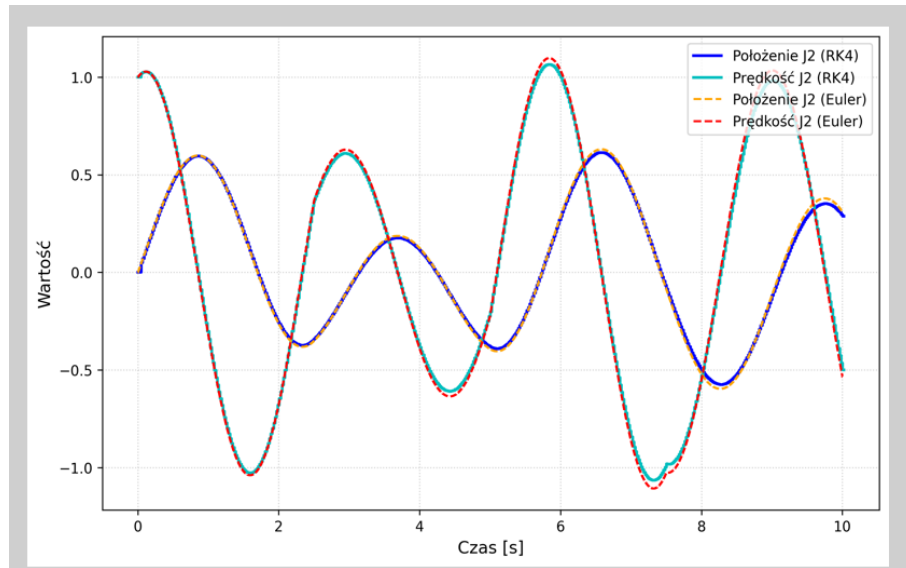


Rysunek 1: Wykres dla układu z silnym tłumieniem

Układ drgającego (bez tłumienia) z parametrami:

- Sygnał wejściowy: Prostokątny
- Amplituda: 1
- Okres: 5
- Czas trwania symulacji: 10
- Rozmiar kroku: 0.05

- Wartość J1: 1
- Wartość J2: 1
- Liczba zębów przekładni 1 (n1): 1
- Liczba zębów przekładni 1 (n2): 1
- Sztywność (k): 9
- Współczynnik tłumienia (b): 0



Rysunek 2: Wykres dla układu drgającego

Testy przeprowadzono dla celowo dobranego dużego kroku czasowego $\Delta t = 0.05$ s, aby uwypuklić różnice w jakości działania algorytmów.

W przypadku wymuszenia prostokątnego układ wykazuje klasyczną odpowiedź na skok jednostkowy ze skończonym czasem trwania. Testy ukazują, iż metoda Eulera wykazuje tendencję do destabilizacji i zawyżania odpowiedzi drgającej, wyraźnie zauważalne w teście dla układu drgającego, co wynika bezpośrednio z faktu, że jej obszar stabilności w płaszczyźnie zespolonej nie obejmuje w pełni osi urojonej dla układów słabo tłumionych. Metoda RK4 generuje krzywe całkowicie stabilne i zbieżne z teorią fizyczną.

6 Wnioski końcowe

Wszystkie założenia i cele postawione przed projektem zostały zrealizowane. Kluczowe spostrzeżenia wynikające z przeprowadzonych badań symulacyjnych:

- **Wyprowadzony model zredukowany** poprawnie oddaje fizykę układu. Koncepcja zredukowanego momentu bezwładności J_{eq} znacząco upraszcza implementację obliczeniową.
- **Metoda Eulera**, ze względu na lokalny błąd pierwszego rzędu, wymaga stosowania drastycznie mniejszych kroków integracji (np. $\Delta t \leq 0.001$ s). Przy większych krokach czasowych wprowadza do układu sztuczną energię numeryczną, doprowadzając do zafałszowania wyników lub rozbiegania symulacji.
- **Metoda RK4** okazała się w pełni stabilna i dokładna, nawet przy relatywnie zgrubej dyskretyzacji czasowej.

Literatura

- [1] Tom Schimansky, *CustomTkinter – A modern UI-library for tkinter in Python*, <https://customtkinter.tomschimansky.com/>